

Service slicing in SDN: garanzie di QoS su un collegamento condiviso

Network and Cloud Infrastructures · Prof. Giorgio Ventre · A.A. 2025/2026

Claudio Caccaviello · matricola DE9000138

Obiettivo. Verificare sperimentalmente se un controller SDN possa garantire gli obiettivi di servizio del traffico sensibile alla latenza mentre un flusso di massa satura lo stesso collegamento. Il lavoro confronta due configurazioni identiche — una con accodamento differenziato programmato via OpenFlow, una con accodamento unico — su diciotto esecuzioni automatizzate a tre livelli di carico.

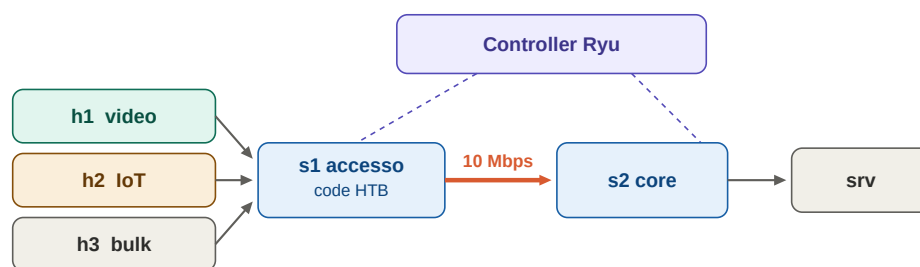
1. Il problema

In una rete a commutazione di pacchetto più flussi condividono lo stesso collegamento; quando la domanda istantanea supera la capacità, il modello best-effort risolve la contesa con una coda FIFO servita in ordine di arrivo. Il difetto emerge quando i flussi hanno requisiti diversi, come osservato al Cap. 2 degli appunti: un trasferimento di file può ritardare una conversazione VoIP per il solo fatto di essere arrivato prima.

La disparità è strutturale. Il flusso di massa **non trae quasi alcun beneficio** dalla bassa latenza che sottrae — il suo dato serve solo alla fine — mentre la comunicazione interattiva la usa interamente. Il danno inflitto è sproporzionato al vantaggio ottenuto.

Il flusso TCP non è aggressivo per difetto di implementazione, ma per progetto: non conoscendo la capacità del percorso, la determina sperimentalmente aumentando la finestra di congestione finché non rileva una perdita. **Per accorgersi di avere esagerato deve prima riempire il buffer**, e nel frattempo chiunque altro attraversi quel punto ne subisce il ritardo di accodamento. Il fenomeno è noto come bufferbloat. Va aggiunto che TCP misura la propria condizione, non quella altrui: raggiunti 9,5 Mbps su un collegamento da 10 considera la situazione ottimale, senza alcun modo di sapere che una videochiamata sta soffrendo. Il controllo di congestione è locale al singolo flusso; nessuno possiede la visione d'insieme. **È precisamente il vuoto che SDN colma.**

2. Architettura



Tratteggio viola: canale OpenFlow (TCP 6653). Freccia arancione: collo di bottiglia, unico punto di contesa.

La topologia è volutamente minimale e presenta un **unico collo di bottiglia**. La scelta è metodologica: con più punti di congestione non sarebbe possibile attribuire con certezza un effetto misurato alla sua causa. I collegamenti di accesso restano quindi sovradimensionati, con solo 5 ms di ritardo e 1 ms di jitter.

2.1 Classi di traffico e code

Slice	Profilo generato	Requisito	Coda	Minimo garantito
video	UDP porta 5004 (RTP), 2 Mbps costanti	latenza e jitter bassi	1	4 Mbps
IoT	UDP porta 1883 (MQTT), 0,5 Mbps a burst	affidabilità	2	2 Mbps
bulk	TCP porta 80 (HTTP), greedy	throughput	0 (default)	1 Mbps
sonda	ICMP verso il server	—	1 (con il video)	—

La somma dei minimi garantiti (7 Mbps) è **inferiore** alla capacità del collegamento. La scelta è deliberata: HTB è work-conserving e ridistribuisce la banda inutilizzata a chi la richiede, cosicché il flusso bulk ottiene in pratica molto più del proprio minimo. Assegnare in garanzia l'intera capacità avrebbe prodotto un partizionamento rigido — multiplexing a divisione fissa, non slicing — con spreco di capacità a ogni slice inattivo.

2.2 Il controller

Il controller comunica con lo switch installandovi delle **regole**. Ogni regola ha due parti: le condizioni che un pacchetto deve soddisfare per esserne catturato, e le azioni da applicargli. Questa è la regola che realizza lo slice video:

Regola per lo slice video	Significato
condizioni	
eth_type = 0x0800	è un pacchetto IP
ipv4_dst = 10.0.0.100	diretto al server
ip_proto = 17	trasportato da UDP
udp_dst = 5004	sulla porta del video
azioni	
SetQueue(1)	assegnalo alla coda 1
Output(4)	inoltralo verso il collo di bottiglia

L'ordine delle azioni non è indifferente: l'assegnazione della coda deve precedere l'inoltro, altrimenti il pacchetto uscirebbe prima di essere stato assegnato. Le condizioni vanno inoltre dichiarate in ordine gerarchico, poiché OpenFlow non consente di specificare la porta UDP senza aver prima indicato che si tratta di traffico UDP, né questo senza aver indicato che si tratta di traffico IP.

Le quattro regole (una per slice) vengono installate **all'avvio**, appena lo switch si collega e prima che transiti un solo pacchetto. Una prima versione le installava invece alla comparsa del primo pacchetto di ciascun flusso, e non funzionava: la regola generica di inoltro, installata dalla connessione di controllo di iperf3, catturava tutto il traffico verso il server, cosicché i pacchetti UDP non raggiungevano più il controller e restavano non classificati. L'installazione all'avvio è anche concettualmente più corretta: la politica esiste perché è stata decisa, non perché qualcuno ha generato traffico.

Va sottolineato che il controller **non manipola i pacchetti**: programma un comportamento, e da quel momento lo switch opera in autonomia. Spegnendolo, il traffico continuerebbe a essere trattato correttamente. Parallelamente, il controller interroga lo switch ogni secondo per raccogliere le statistiche; poiché i contatori OpenFlow sono cumulativi, il throughput istantaneo è ottenuto come loro derivata.

3. Metodo sperimentale

Diciotto esecuzioni automatizzate: due scenari, tre livelli di carico del flusso bulk (5, 10 e 20 Mbps), tre ripetizioni ciascuno. Lo script orchestra ogni combinazione avviando il controller nella modalità corretta, costruendo la topologia, generando il traffico e raccogliendo i risultati. La topologia non è ridefinita nello script ma importata da `topology.py`, perché i percorsi manuale e automatico non possano divergere.

Scelta metodologica	Motivazione
Stesso meccanismo di limitazione nei due scenari	Anche la configurazione di riferimento impiega una QoS HTB, ma con una coda sola. Il limite di 10 Mbps è imposto dallo stesso identico meccanismo e l'unica variabile che cambia è la differenziazione.
La sonda ICMP viaggia nella coda del video	Lasciata nella coda di default misurerebbe il ritardo sofferto dal bulk anziché quello goduto dal traffico protetto. Classificandola nella coda video diventa una sonda interna allo slice osservato.
Campionamento ogni 200 ms	Cento misure per esecuzione anziché venti: con pochi campioni il 95° percentile coinciderebbe di fatto con il massimo.
La capacità è imposta da OVS, non da TCLink	Entrambi configurano un qdisc HTB sulla stessa interfaccia e si sovrascrivono a vicenda, senza segnalare errori: le code risulterebbero presenti ma inefficaci.

3.1 Verifica preliminare

Prima di automatizzare la campagna, i due scenari sono stati confrontati manualmente. Di seguito l'output del comando ping eseguito sull'host del video mentre tutti e tre i flussi erano attivi e il collegamento saturo. La sola differenza fra le due prove è la presenza della differenziazione del traffico.

```

--- con slicing attivo ---
15 packets transmitted, 15 received, 0% packet loss
rtt min/avg/max/mdev = 9.169/10.568/11.461/0.560 ms

--- senza slicing, stesse condizioni ---
15 packets transmitted, 13 received, 13.3% packet loss
rtt min/avg/max/mdev = 474.948/1160.963/1912.709/496.001 ms

```

I quattro valori riportati da ping sono, nell'ordine, il tempo di andata e ritorno minimo, medio, massimo e la deviazione standard. Quest'ultima è particolarmente indicativa: 0,56 ms contro 496 ms misura la stabilità del ritardo, ossia il jitter.

4. Risultati

Carico bulk	Scenario	RTT medio	RTT p95	Perdita sonda	Scarti coda bulk
-------------	----------	-----------	---------	---------------	------------------

5 Mbps	slicing	10,52 ms	11,90 ms	0 %	0
5 Mbps	baseline	47,30 ms	102,27 ms	0 %	0
10 Mbps	slicing	10,53 ms	12,00 ms	0 %	492
10 Mbps	baseline	933,09 ms	1979,00 ms	7,3 %	615
20 Mbps	slicing	10,53 ms	11,87 ms	0 %	486
20 Mbps	baseline	865,00 ms	1970,00 ms	6,3 %	594

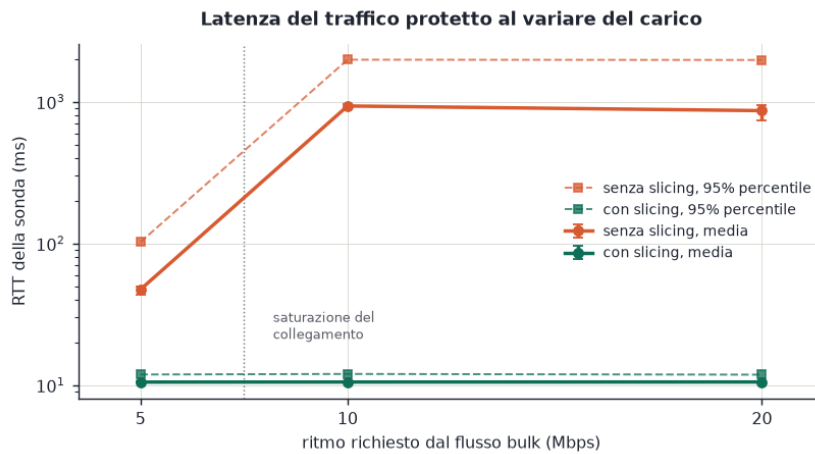


Fig. 1 — Latenza del traffico protetto al variare del carico offerto. Scala logaritmica; le barre di errore coprono le tre ripetizioni.

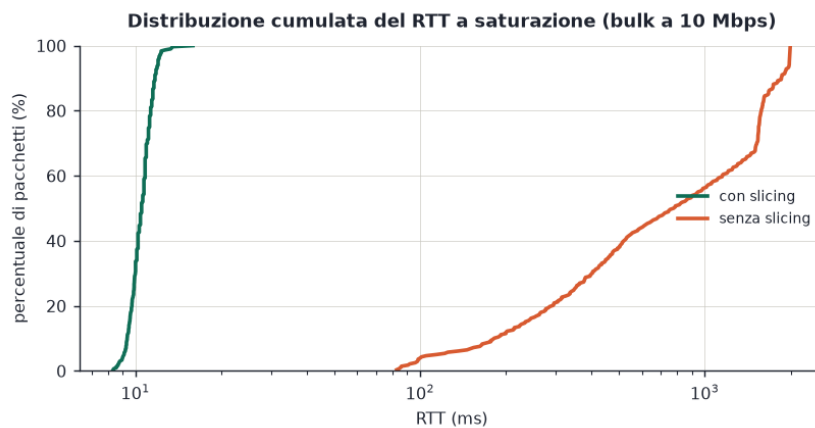


Fig. 2 — Distribuzione cumulata dei campioni di RTT a saturazione. Le due distribuzioni non presentano alcuna sovrapposizione.

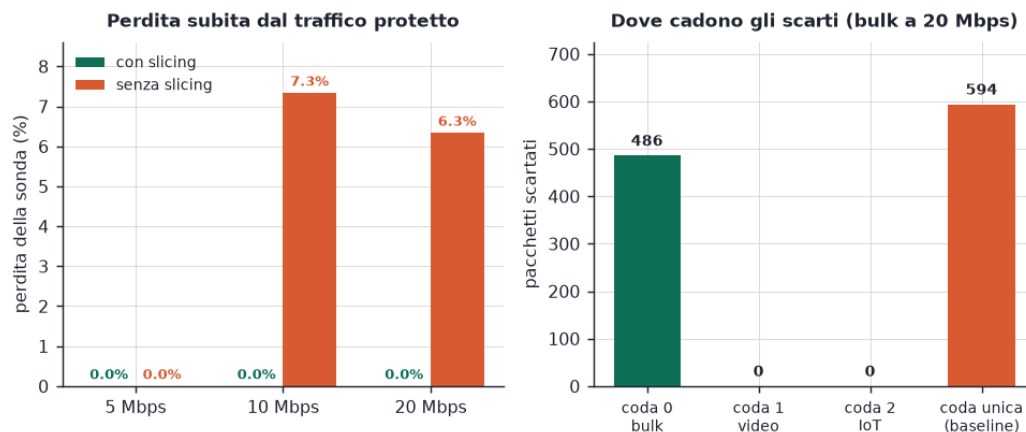


Fig. 3 — A sinistra: perdita subita dal traffico protetto. A destra: distribuzione degli scarti fra le code, a carico massimo.

4.1 Considerazioni

Slicing insensibile al carico

Il dato più significativo non è che la latenza sia bassa, ma che sia **costante**. Su nove esecuzioni con carico che varia da metà capacità a doppio sovraccarico, i valori restano fra 10,52 e 10,53 ms di media e sempre entro 12,1 ms al 95° percentile. Le barre di errore in Fig. 1 sono così piccole da risultare invisibili. Lo slice protetto non è semplicemente avvantaggiato: è *isolato*

da ciò che accade negli altri slice.

Soglia netta, non degrado graduale

A 5 Mbps la domanda complessiva ($5 + 2 + 0,5 = 7,5$ Mbps) resta sotto la capacità e il degrado è contenuto. Superata la saturazione il comportamento cambia qualitativamente: la latenza cresce di un fattore novanta e compaiono perdite prima assenti. Passando da 10 a 20 Mbps la situazione non peggiora ulteriormente (933 contro 865 ms, con le ripetizioni sovrapposte): una volta pieno, il buffer non può riempirsi di più. Ne è conferma il 95° percentile, stabile intorno a 1975 ms in tutte le esecuzioni sature — quel valore non dipende dal carico, è il ritardo di accodamento massimo consentito dalla dimensione del buffer, una costante strutturale del sistema.

Distribuzioni disgiunte

La Fig. 2 mostra un risultato che le sole medie non rivelano: **nessun valore di RTT compare in entrambi gli scenari**. Lo slicing resta fra 9 e 16 ms, il baseline parte da 80 ms. Il peggior pacchetto protetto è cinque volte migliore del migliore pacchetto non protetto. Anche la forma è informativa: la curva dello slicing è quasi verticale, dunque tutti i pacchetti subiscono la stessa latenza e il jitter è trascurabile — ed è il jitter, prima ancora della latenza assoluta, a rendere inutilizzabile una comunicazione interattiva. La curva del baseline si distende su due decadi, con un gradino verticale intorno ai 1900 ms: un terzo dei pacchetti subisce il ritardo massimo producibile.

Considerazione finale: non conta quanto si perde, ma chi paga

Nello scenario con slicing gli scarti sono **tutti** concentrati nella coda del traffico bulk: si tratta di segmenti TCP, che il protocollo ritrasmette senza che l'utente se ne accorga. Le code del video e della telemetria non registrano alcuna perdita in nessuna delle nove esecuzioni. Il baseline, a parità di traffico e di capacità, ne scarta anche in quantità maggiore (594 contro 486) e soprattutto li distribuisce indiscriminatamente su tutti i flussi, sonda compresa. Una valutazione basata sul solo tasso di perdita aggregato non avrebbe colto la differenza: **contano le metriche per classe, non gli aggregati**.

5. Riproducibilità

File	Ruolo
topology.py	Topologia Mininet, creazione delle code HTB su Open vSwitch. <code>--qos slicing none</code> seleziona lo scenario.
slicing_controller.py	Applicazione Ryu: policy proattiva, apprendimento MAC, monitoraggio su CSV. Le variabili <code>SLICING_MODE</code> e <code>SLICING_CSV</code> ne governano il comportamento.
run_experiments.py	Campagna automatica: orchestra le 18 esecuzioni e produce <code>riepilogo.csv</code> e <code>campioni_rtt.csv</code> .
plot.py	Genera le tre figure a partire dai risultati.
<pre>\$ sudo python3 run_experiments.py # campagna completa, ~10 minuti \$ python3 plot.py # figure in figure/</pre>	

Ambiente: VM Ubuntu 20.04, Mininet 2.3.0, Open vSwitch, Ryu 4.34, OpenFlow 1.3, iperf3.

6. Limiti e sviluppi

La topologia presenta un unico punto di contesa: la scelta favorisce la chiarezza interpretativa ma non riproduce una rete reale con congestione distribuita. Il traffico è generato sinteticamente a ritmo costante e non riproduce la variabilità del traffico applicativo autentico. Le tre ripetizioni per configurazione consentono di verificare la consistenza dei risultati ma non di costruire intervalli di confidenza rigorosi.

Il limite principale è che la politica di QoS è **statica**: viene installata all'avvio e non si adatta alle condizioni osservate. Va però notato che il comportamento work-conserving di HTB realizza già una forma di allocazione dinamica, poiché la capacità non utilizzata da uno slice viene automaticamente resa disponibile agli altri.

L'estensione naturale è il **controllo in retroazione**: un controller che monitora le statistiche già raccolte, rileva la violazione di una soglia di servizio e modifica a tempo di esecuzione i parametri delle code. È il caso in cui la visione globale del piano di controllo diventa indispensabile, perché nessun dispositivo singolo — e nessun algoritmo di congestione locale al flusso — potrebbe prendere quella decisione da solo.